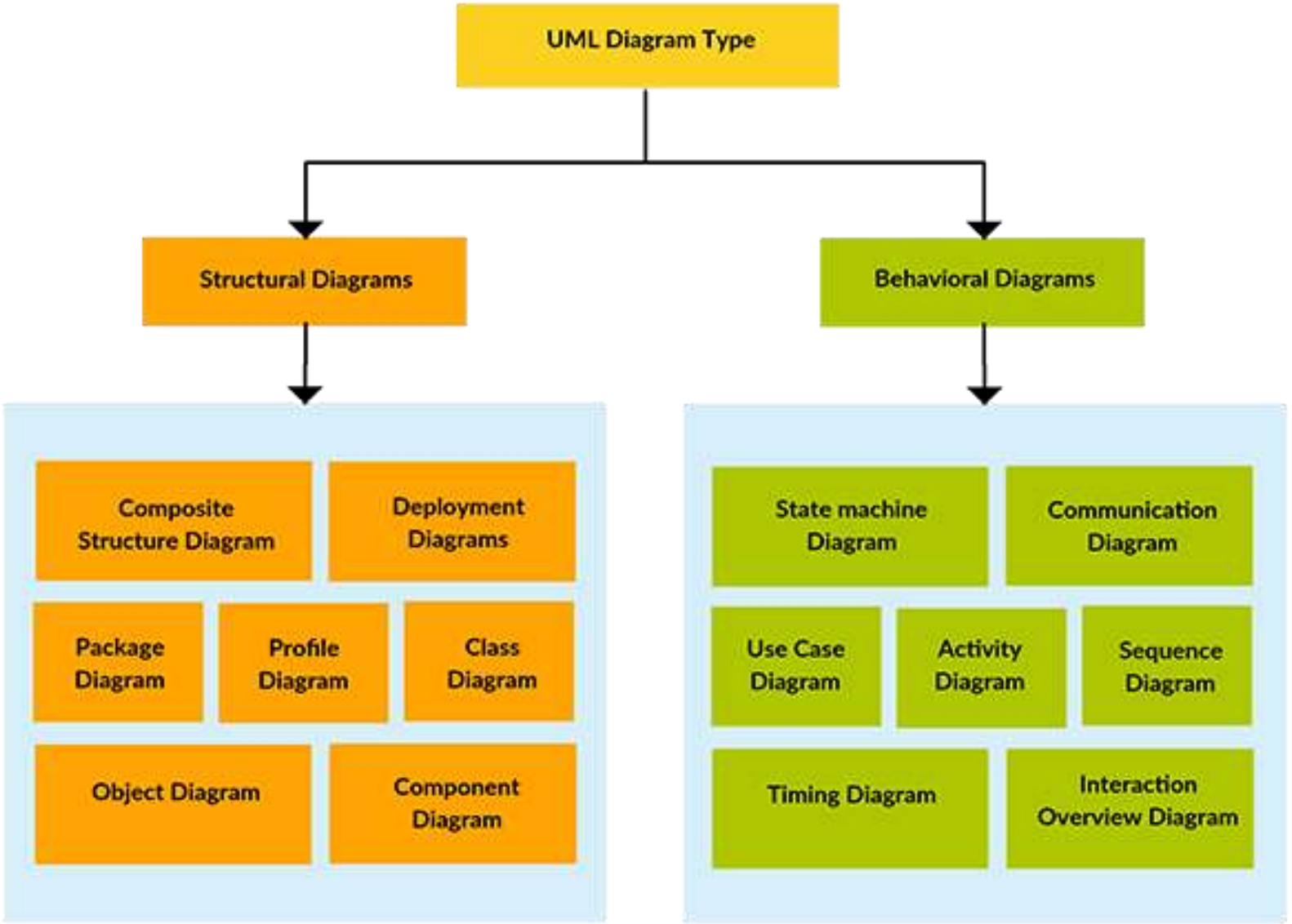
ICT 1306
OBJECT ORIENTED PROGRAMMING
UML

TC Irugalbandara
KHA Hettige

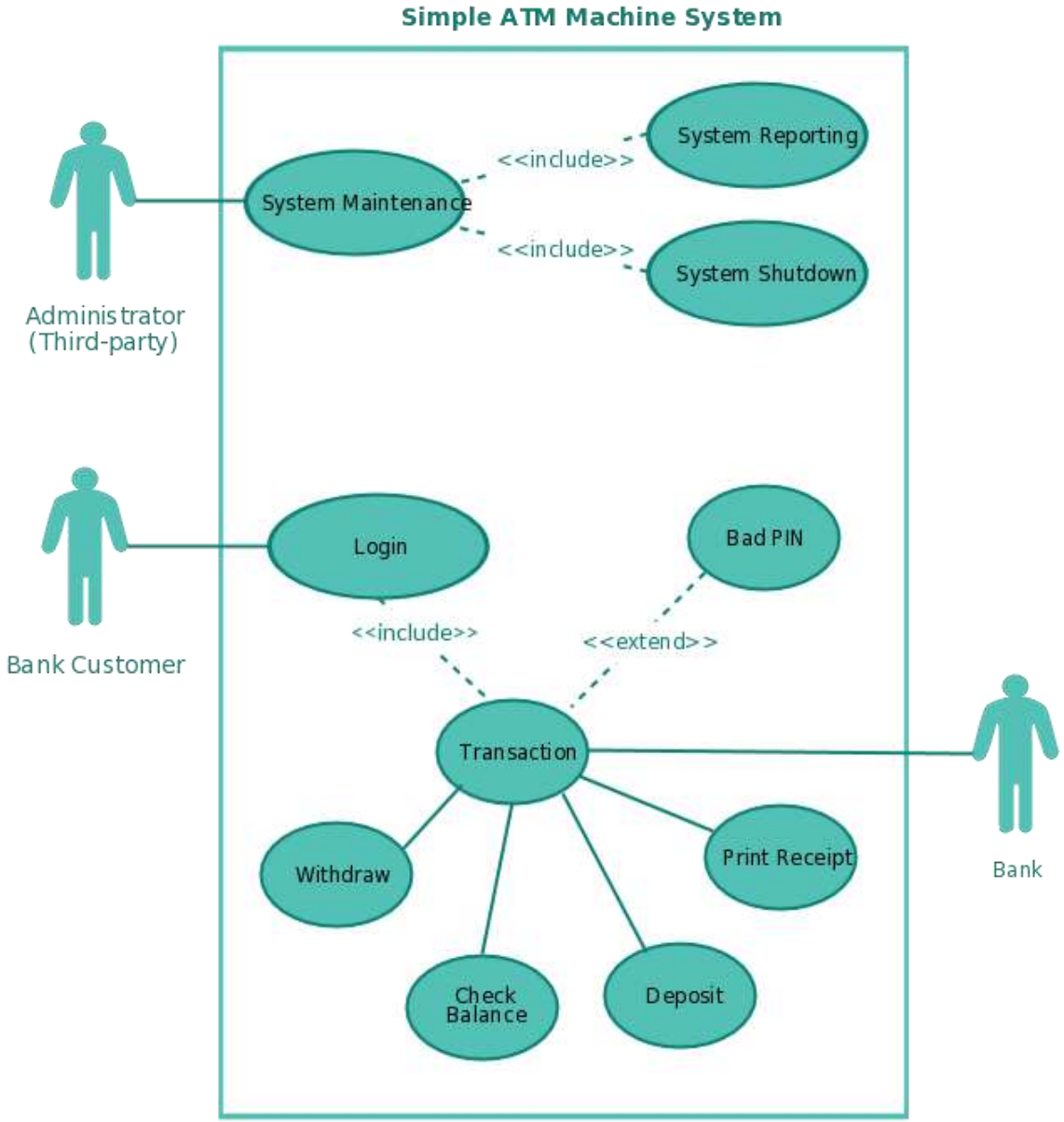
Introduction

- UML stands for Unified Modeling Language.
- The UML is a general purpose visual modeling language that is used to
  - ✓ specify,
  - ✓ visualize,
  - ✓ construct and
  - ✓ documents the artifacts of a software system.
- A graphical way of describing software systems.
- UML is not a programming language.


UML Diagram Type
Structural Diagrams
Behavioral Diagrams
Composite Structure Diagram
Deployment Diagrams
Package Diagram
Profile Diagram
Class Diagram
Object Diagram
Component Diagram
State machine Diagram
Communication Diagram
Use Case Diagram
Activity Diagram
Sequence Diagram
Timing Diagram
Interaction Overview Diagram

Use Case

- Use case enable to visualize the different types of roles (known in the UML as an actor) involved in a system and how those roles interact with the system.


Simple ATM Machine System
System Reporting
<<include>>
System Maintenance
<<include>>
System Shutdown
Administrator
(Third-party)
Bad PIN
Login
<<include>>
<<extend>>
Bank Customer
Transaction
Bank
Withdraw
Print Receipt
Check
Balance
Deposit

Use case diagram objects

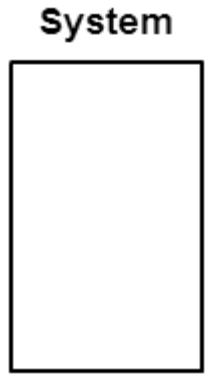
- Actor :
  - ✓ Any entity that performs a role in one given system.
  - ✓ This could be a person, organization or an external system .
  - ✓ Usually drawn like skeleton shown below.

Use case diagram objects

- Use Case :
  - ✓ Represents a function or an action within the system.
  - ✓ Its drawn as an oval and named with the function

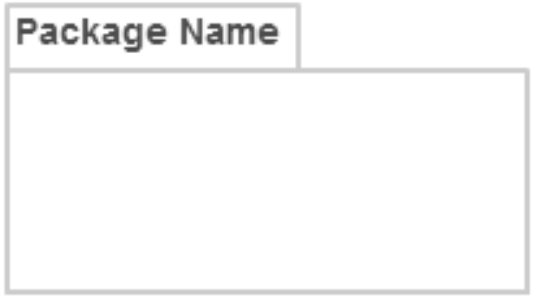
Use case diagram objects

- System:
  - ✓ Define the scope of the use case and drawn as a rectangle.
  - ✓This an optional element but useful when your visualizing large systems.



Use case diagram objects

- Package:
  - ✓ Used to group together use cases.



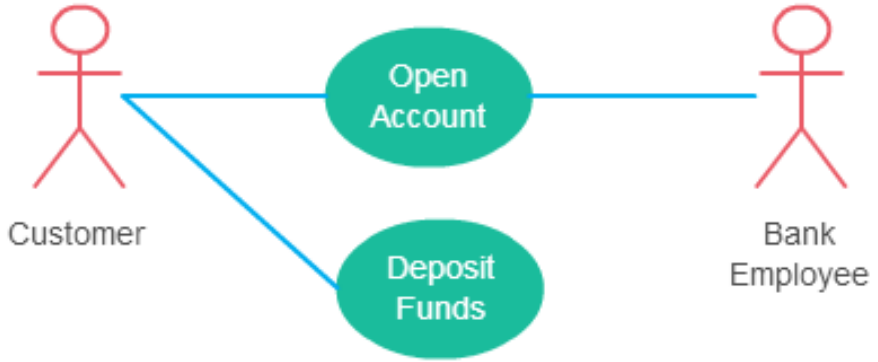
Relationships in Use Case Diagrams

- Association between an actor and a use case.
- Generalization of an actor.
- Extend relationship between two use cases.
- Include relationship between two use cases.
- Generalization of a use case.

Association between an actor and a use case

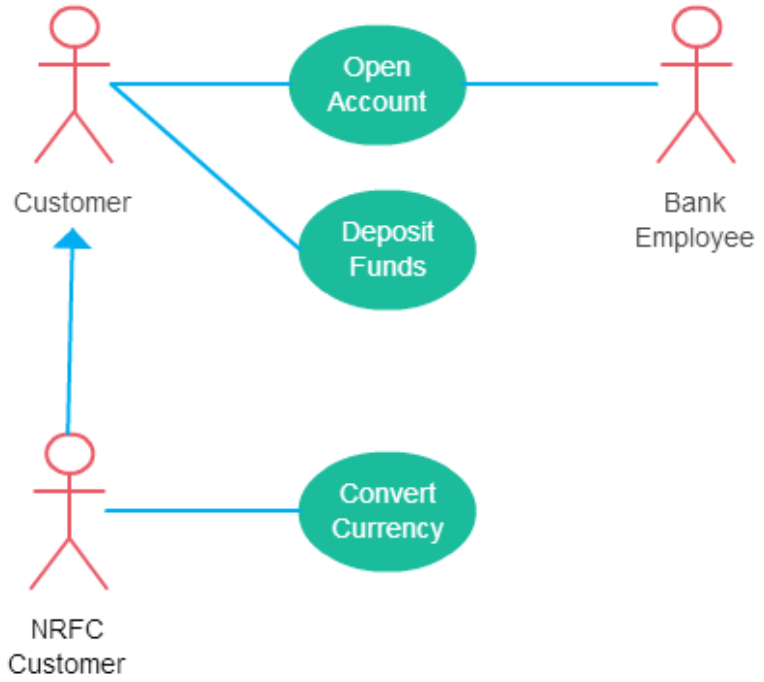
This is straightforward and present in every use case diagram. Following things must be note.

- An actor must be associated with at least one use case.
- An actor can be associated with multiple use cases.
- Multiple actors can be associated with a single use case.



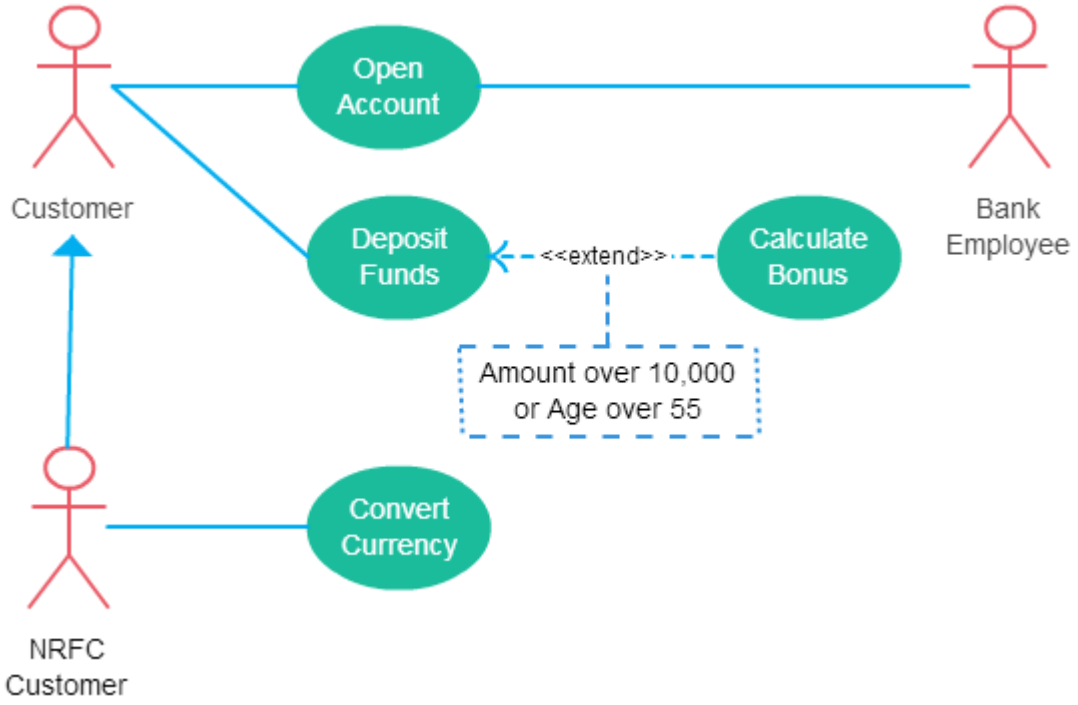
Generalization of an Actor

- One actor can inherit the role of an other actor.
- The descendant inherits all the use cases of the ancestor.
- The descendant have one or more use cases that are specific to that role.



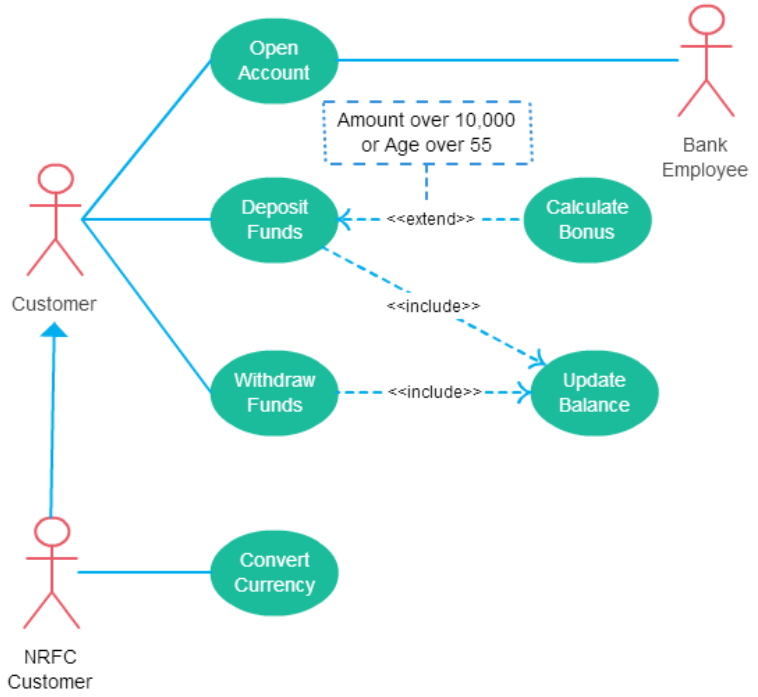
Extend Relationship Between Two Use Cases

- It extends the base use case and adds more functionality to the system.
- The extending use case is dependent on the extended (base) use case.
- The extending use case is usually optional and can be triggered conditionally.
- Base use case should be independent and must not rely on the behavior of the extending use case.



Include Relationship Between Two Use Cases

- The behavior of the included use case is part of the including (base) use case.
- The main reason for this is to reuse the common actions across multiple use cases.
- The base use case is incomplete without the included use case.
- The included use case is mandatory and not optional.



Generalization of a Use Case

- The behavior of the ancestor is inherited by descendant.
- This is used when there are common behavior between two use cases.

Use Case Scenario

- A use case has multiple “paths” that can be taken by any user at any one time.
- A use case scenario is a single path through the use case.

Withdraw money from an ATM

Primary actors: Customer
ATM Technician
Bank

Preconditions: Network connection is active
ATM has available cash

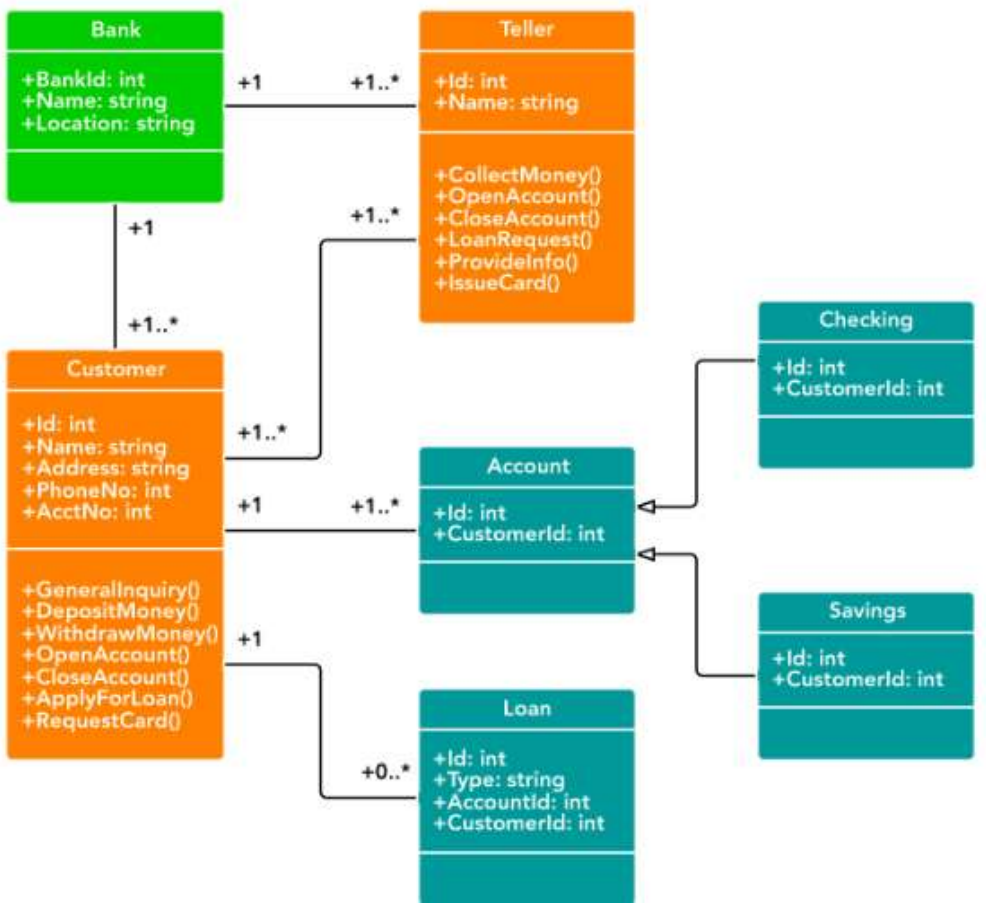
Basic flow of events:
1. Bank customer inserts debit card and enters PIN.
2. Customer is validated.
3. ATM displays actions available on ATM unit. Customer selects Withdraw Cash.
4. ATM prompts account.
5. Customer selects account.
6. ATM prompts amount.
7. Customer enters desired amount.
8. Information sent to Bank, inquiring if sufficient funds/allowable withdrawal limit.
9. Money is dispensed and receipt prints.

Alternative flows

2a. Customer is not validated.
  2a1. ATM displays error message.
7a. Customer selects invalid amount.
  7a1. ATM prompts user to re-enter valid amount.
8a. Customer has insufficient funds.
  8a1. ATM displays error message.
  8a2. ATM shows available withdrawal limit, redirects to step 6.
9a. ATM has insufficient cash.
  9a1. ATM Technician is alerted.
  9a2. ATM displays error message and phone number to call.
9b. Cash gets stuck in dispensing.
  9b1. ATM displays error message.

Class Diagram

- Describes the structure of a system by showing its classes, attributes and operations of those classes and the relationship between those classes.



Basic Class Diagram Symbols and Notations

- Classes
  - ✓ Illustrate classes with rectangles divided into components.
  - ✓ Place the name of the class in the first partition (centered, bolded and capitalized).
  - ✓ List the attributes in the second partition (left-aligned, not bolded and lowercase).
  - ✓ Write operations into the third.

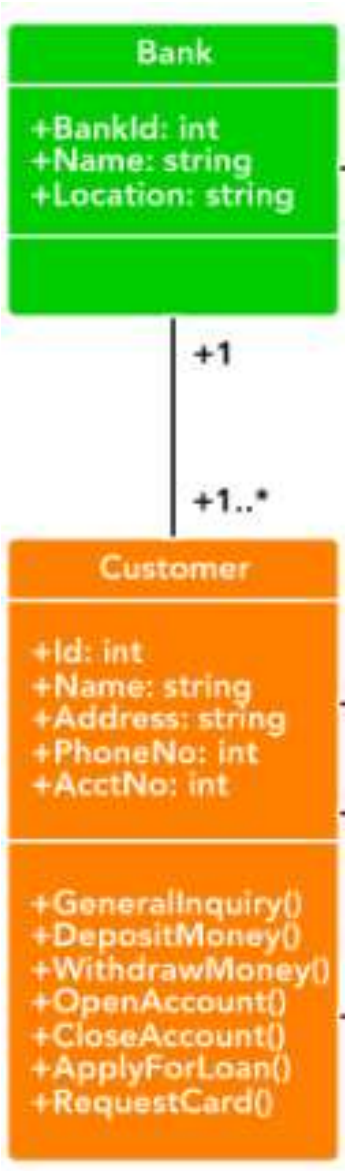
Basic Class Diagram Symbols and Notations

- Visibility
  - ✓ Use visibility markers to signify who can access the information contained with in a class.

Marker	Visibility
+	Public
-	Private
#	Protected
~	Package

Basic Class Diagram Symbols and Notations

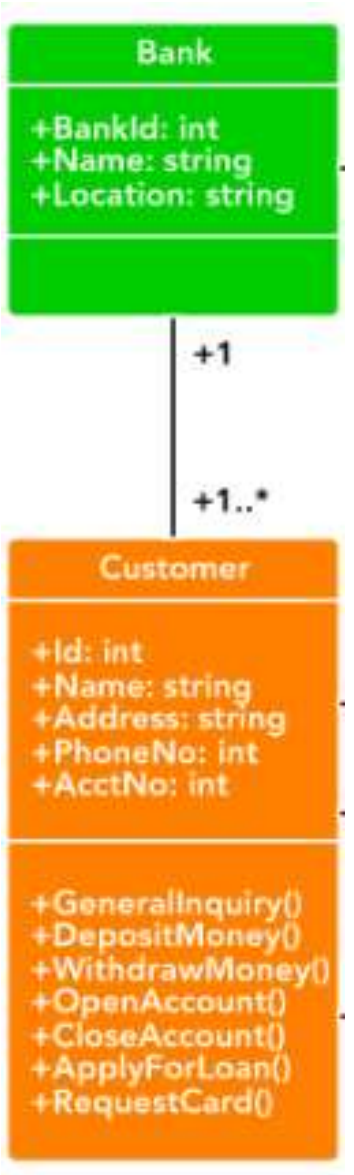
- Association
  - ✓ Any logical connection or relationship between classes.



Basic Class Diagram Symbols and Notations

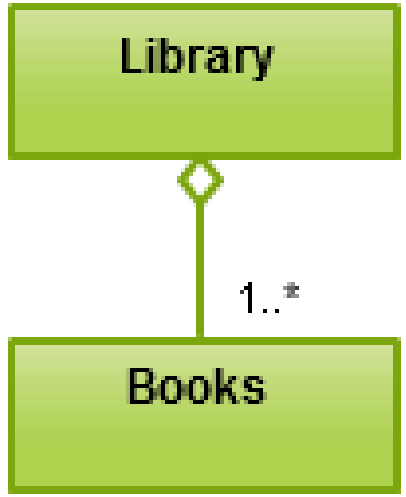
- Multiplicity (Cardinality)
  - ✓ Place multiplicity notations near the ends of an association.
  - ✓These symbols indicate the number of instances of one class linked to one instance of the other class

Indicator	Meaning
0..1	Zero or one
1	One only
0..*	0 or more
1..* *	1 or more
n	Only n(where n>1)
0..n	Zero to n (where n>1)
1..n	One to n (where n>1)



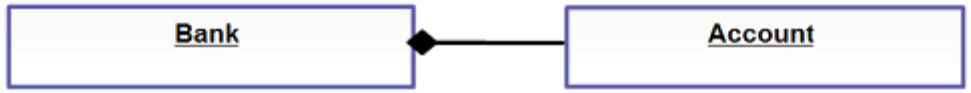
Basic Class Diagram Symbols and Notations

- Aggregation
  - ✓ When a class is formed as a collection of other classes, it is called an aggregation relationship between these classes.
  - ✓ It is also called a "has a" relationship.
  - ✓ In aggregation, the contained classes are not strongly dependent on the life cycle of the container.
  - ✓ To render aggregation in a diagram, draw a line from the parent class to the child class with a diamond shape near the parent class.



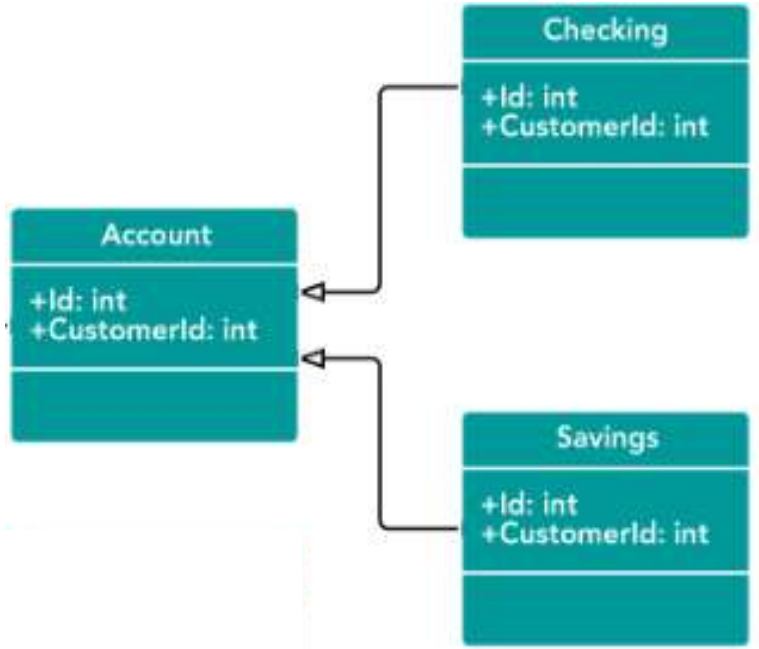
Basic Class Diagram Symbols and Notations

- Composition
  - ✓ Variation of the aggregation relationship.
  - ✓ In composition contained class will be obliterated when the container class is destroyed.
  - ✓ To indicate composition relationship, use a directional line connecting the two classes with a filled diamond shape adjacent to the container class and the directional arrow to the contained class.



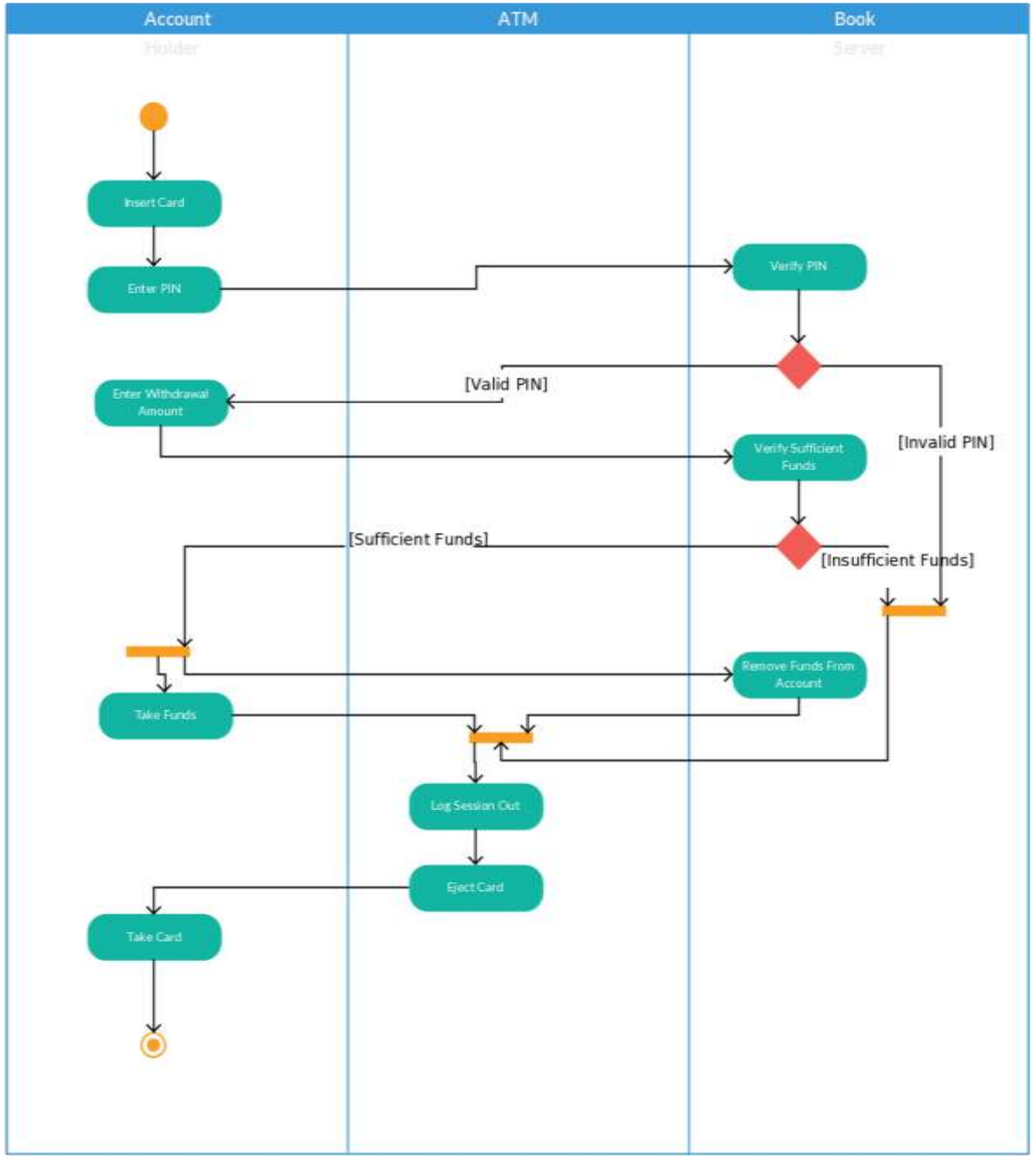
Basic Class Diagram Symbols and Notations

- Generalization / Inheritance
  - ✓ Known as an "is a" relationship since the child is a type of the parent class.
  - ✓The child classes "inherit" the common functionality defined in the parent class.



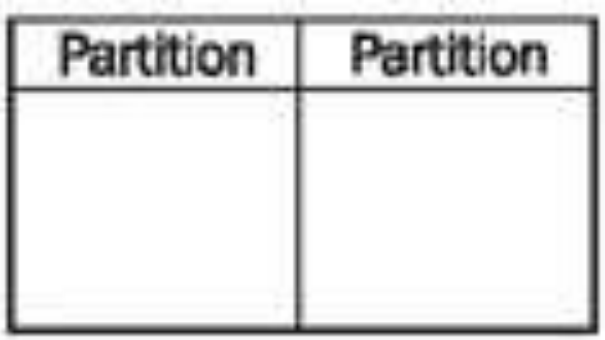
Activity Diagram

- Activity diagram is basically a flow chart to represent the flow from one activity to another activity.
- The activity can be describe as an operation of the system.


Account
Holder
ATM
Book
Server
Insert Card
Enter PIN
Verify PIN
[Valid PIN]
Enter Withdrawal Amount
Verify Sufficient Funds
[Invalid PIN]
[Sufficient Funds]
[Insufficient Funds]
Remove Funds From Account
Take Funds
Log Session Out
Eject Card
Take Card

Basic Activity Diagram Notations and Symbols

- Activity Partition
  - ✓ The individual elements of an activity diagram can be divided into individual areas or "Partitions".



Basic Activity Diagram Notations and Symbols

- Initial State or Start Point
  - ✓ A small filled circle followed by an arrow represent the initial action sate or the start point for any activity.

Start Point/Initial State

Basic Activity Diagram Notations and Symbols

- Action State
  - ✓ An action is an individual step with in an activity.

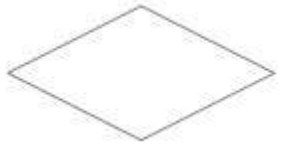
Activity

Basic Activity Diagram Notations and Symbols

- Action Flow
  - ✓ The transitions from one action to another.
  - ✓ Usually drawn with an arrowed line.

Basic Activity Diagram Notations and Symbols

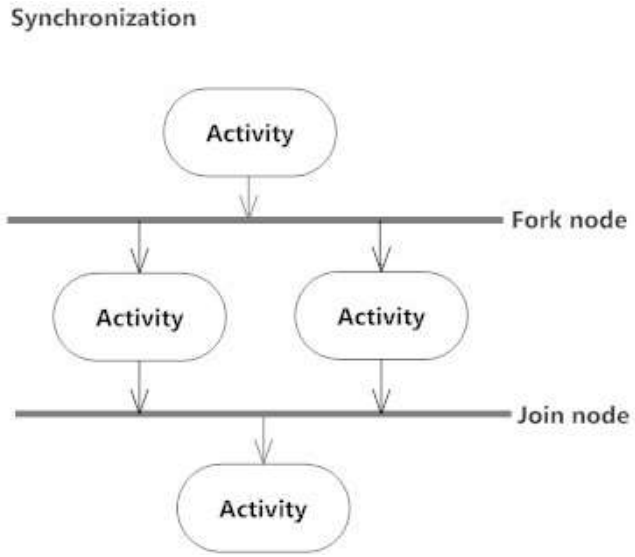
- Decisions
  - ✓ A diamond represent a decision with alternative path.
  - ✓When an activity requires a decision prior to moving on to the next activity, add a diamond between the two activities.
  - ✓The outgoing alternates should ne labeled with a condition.



Decision Symbol

Basic Activity Diagram Notations and Symbols

- Synchronization
  - ✓ A fork node is used to split a single incoming flow into multiple concurrent flows. It is represented as a straight, slightly thicker line.
  - ✓ A join node joins multiple concurrent flows back into a single outgoing flow.



Basic Activity Diagram Notations and Symbols

- Final State or End Point
  - ✓ An arrow pointing to a filled circle nested inside another circle represents the final action state or represent completion of a workflow.

End Point Symbol

UML Tools

- Smartdraw
- Visual paradigm
- Creatly

Thank You !